Does In-Loop LLM Review Make Patches Correct, or Only Make Them Pass? A Research Gap in Repository-Level LLM4APR

Research Gap Statement · Lab 5, Research Methods (Week 5) · Rifqi Wibisono (魏副棋), L25020014 · Bottleneck selected: **patch correctness** (with cost), from the MAGIS paper [2] and the LLM4APR review [1]

1. Background and state of the art

LLM-based automated program repair (LLM4APR) grew from one study in 2020 to 189 by September 2025, and its frontier has moved from single-function bugs to repository-level issue resolution [1]. The standard testbed, SWE-bench, gives a model a real GitHub issue and a full Python repository. It counts the issue as *resolved* when the model's patch makes the developer's hidden tests pass [3]. Directly prompted models resolve almost nothing in this setting: GPT-4 resolves 1.74% even when given the files to edit. The field responded with agentic pipelines. MAGIS [2] divides the work among four LLM roles: Manager, Repository Custodian, Developer, and a Quality Assurance (QA) Engineer that reviews every code change and returns it for revision until it approves. MAGIS reports 13.94% resolution. In-loop review has since spread to other systems [9].

2. The specific unresolved gap

All these gains are measured by one oracle: the tests that shipped with the developer's fix. Tests are an incomplete specification, and APR research has known since Qi et al. [7] that test-passing (*plausible*) patches are often wrong: GenProg's 55 reported fixes contained 2 correct ones. The problem persists at repository scale. On SWE-bench Verified, the human-filtered 500-issue subset, the full test suite fails 7.8% of patches counted as resolved. Differential testing shows 29.6% behave differently from the developer's patch, and an estimated 11% of plausible patches are incorrect, inflating reported resolution by 6.4 points [4]. Test augmentation exposed 345 mislabelled patches [5] and cut top agents' resolved rates by 4.2–9.0% [6].

Against this, MAGIS credits its QA reviewer with +3.31 points with issue hints and +1.57 without [2]. These gains are smaller than the measured inflation and are measured by the same oracle that produces it.

Gap: whether in-loop LLM review makes patches more correct, or mostly turns failing patches into plausible-but-incorrect ones, has not been measured. None of the six LLM-based patch-correctness assessors in the LLM4APR review was evaluated on repository-level issue patches [1, §8.3].

3. Why existing approaches fail (root cause)

The reviewer sees text, not behaviour. MAGIS's QA Engineer is the same model reading a diff, so it inherits the generator's blind spots. Olausson et al. found self-repair is bottlenecked by exactly this: a model's ability to diagnose its own code [8].

The errors that survive testing are largely invisible in a diff. 27.3% of behaviourally divergent patches change *more* behaviour than the developer's fix, and faulty supplementary changes cause 36.4% of the patches confirmed incorrect [4]. A side effect on an untested path neither appears in the diff nor is exercised by an issue-specific test.

Execution-aware review checks the intended change, not unintended ones. SpecRover's reviewer and Agentless's validation run reproduction and regression tests [9, 11]. These confirm the reported symptom is gone and already-tested behaviour is preserved, but say nothing about untested behaviour the patch altered.

Review has no correctness feedback signal. Reviewers are designed and scored against the same weak tests, so pushing a patch to *pass* rather than to be *right* is rewarded. The 2026 review-loop preprints (SWE-Review, RETRACE) also report gains on the original tests only.

Cost is unreported. MAGIS gives no token or monetary cost, so whether review pays for itself is unknown, an omission Kapoor et al. identify as distorting agent research [12].

4. Proposed intervention and expected contribution

Intervention: differential review. Before the reviewer decides, the pipeline generates and executes inputs on which the original and the patched repository behave differently. The reviewer must justify each observed difference against the issue text, or reject the patch and return the differentiating input as concrete feedback. This brings DiffTGen's idea of exposing original-versus-patched differences [10] into the LLM review loop. It reuses PatchDiff's call-trace-guided test generator [4], with one change: PatchDiff compares against the developer's patch, which exists only at evaluation time, while differential review compares against the original program, which is available at repair time.

Evaluation. We will run inside MAGIS's released implementation, whose QA step is a configuration switch and which records the pre-review patch. The benchmark is SWE-bench Verified, with one fixed low-cost model, the files to edit

supplied (holding localisation constant) and issue hints disabled (they can leak solutions). Each instance yields paired patches under no review, text-only review and differential review. Each patch is scored by the original tests and by a strengthened oracle: the full developer test suite, PatchDiff against the developer's patch, augmented tests where released [5, 6], and two-rater manual adjudication of divergent patches.

Expected contributions: (1) the first measurement of how much of in-loop review's gain survives a strengthened oracle; (2) a review mechanism aimed at the failure class tests miss; (3) the cost per correctly resolved issue of each review design. The novelty claim is deliberately narrow: review and differential testing each exist; what is new is their controlled combination, evaluated beyond the benchmark's own tests.

Research questions

Research question	Independent variable(s)	Dependent variable(s)	Hypothesis and analysis
RQ1. To what extent does the resolution gain from text-only in-loop review (MAGIS's QA Engineer) persist when correctness is judged by a strengthened oracle instead of SWE-bench Verified's original tests?	**Review:** none vs text-only. Paired per instance. **Oracle:** original tests vs strengthened oracle.	Resolution rate under each oracle. **Illusory gain:** $\Delta_{\text{original}} - \Delta_{\text{strengthened}}$.	**H1:** $\Delta_{\text{strengthened}} < \Delta_{\text{original}}$. **Analysis:** McNemar's exact test; bootstrap 95% CI on the illusory gain.
RQ2. Does differential review raise the correct-resolution rate and lower the rate of plausible-but-incorrect patches, compared with text-only review?	**Reviewer type:** text-only vs differential.	Correct-resolution rate (strengthened oracle). Share of plausible patches judged incorrect. Reviewer false-approval rate. Reviewer false-rejection rate.	**H2:** differential review lowers the incorrect-plausible share without lowering correct resolution. **Analysis:** McNemar's test; Cohen's κ for the two raters.
RQ3. What does each review design cost per correctly resolved issue, and does differential review's extra test generation and execution pay for its correctness gain?	**Review condition:** none, text-only, differential.	Input tokens, output tokens, US$ and wall-clock time per *correctly* resolved issue (strengthened oracle).	**H3:** differential review has a lower cost per correct resolution than text-only review. **Analysis:** bootstrap CIs; cost–accuracy Pareto plot [12].

Controlled variables in all RQs: model and temperature, instance set, supplied files, hints off, maximum review rounds (2, as in MAGIS's released code), SWE-bench Docker harness.

References

[1] Q. Zhang, C. Fang, Y. Xie, Y. Ma, W. Sun, Y. Yang, Z. Chen. A Systematic Literature Review on Large Language Models for Automated Program Repair. *ACM TOSEM*, 2026. doi:10.1145/3799693

[2] W. Tao, Y. Zhou, Y. Wang, W. Zhang, H. Zhang, Y. Cheng. MAGIS: LLM-Based Multi-Agent Framework for GitHub Issue ReSolution. *NeurIPS*, 2024.

[3] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, K. Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *ICLR*, 2024.

[4] Y. Wang, M. Pradel, Z. Liu. Are "Solved Issues" in SWE-bench Really Solved Correctly? An Empirical Study. *ICSE*, 2026. doi:10.1145/3744916.3764576

[5] B. Yu, Y. Zhu, P. He, D. Kang. UTBoost: Rigorous Evaluation of Coding Agents on SWE-Bench. *ACL*, 2025.

[6] C. Li, Y. Xu, Z. Wang, S. H. Tan, T.-H. Chen. Probe to Generate: Program Variant-Guided Test Augmentation for Repository-Level Repair Benchmarks. *ASE*, 2026 (accepted).

[7] Z. Qi, F. Long, S. Achour, M. Rinard. An Analysis of Patch Plausibility and Correctness for Generate-and-Validate Patch Generation Systems. *ISSTA*, 2015.

[8] T. X. Olausson, J. P. Inala, C. Wang, J. Gao, A. Solar-Lezama. Is Self-Repair a Silver Bullet for Code Generation? *ICLR*, 2024.

[9] H. Ruan, Y. Zhang, A. Roychoudhury. SpecRover: Code Intent Extraction via LLMs. *ICSE*, 2025.

[10] Q. Xin, S. P. Reiss. Identifying Test-Suite-Overfitted Patches through Test Case Generation. *ISSTA*, 2017.

[11] C. S. Xia, Y. Deng, S. Dunn, L. Zhang. Agentless: Demystifying LLM-Based Software Engineering Agents. *Proc. ACM Softw. Eng. (FSE)*, 2025. doi:10.1145/3715754

[12] S. Kapoor, B. Stroebl, Z. S. Siegel, N. Nadgir, A. Narayanan. AI Agents That Matter. *Transactions on Machine Learning Research*.

Preprints cited for currency (not peer-reviewed): R. Wang et al., SWE-Review, arXiv:2607.06065 (2026); C. Li et al., RETRACE, arXiv:2608.08950 (2026).